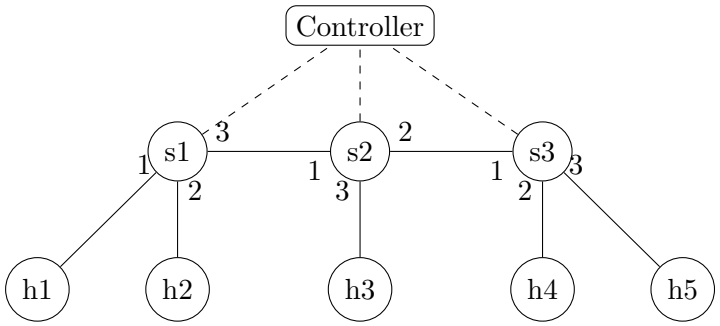
Piattaforme Software per la Rete (Mod. 1) – 02/09/2015

Su ogni foglio consegnato scrivere nome, cognome, matricola e il numero dell'esercizio.

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse.



Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1.

Scrivere tutti i pacchetti che sono inviati su tutti i collegamenti della rete e tra i nodi e il controllore in coseguenza degli eventi seguenti. Scrivere il contenuto delle tabelle openflow prima e dopo ogni domanda.

(a) Invio di un datagramma UDP da h3:123 a h5:88 e relativa risposta.

(b) Invio di un datagramma UDP da h5:123 a h2:100 e relativa risposta.

(c) Apertura di una connessione TCP da h2:123 a h5:80.

Programma 1 (1/2)

```
# intestazione omessa
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                         actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    if (datapath.id == 1) or (datapath.id == 3):
        match = parser.OFPMatch(eth_dst=MAC5)
        actions = [ parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

Programma 1 (2/2)

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']

    pkt = packet.Packet(msg.data)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp)
    pkt_tcp = pkt.get_protocol(tcp.tcp)

    if (pkt_udp is not None) and (pkt_ipv4.dst == IP2):

        pkt_udp.dst_port = pkt_udp.dst_port + 1
        pkt.serialize()

        data = pkt.data
        actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
        out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                                  in_port=in_port, actions=actions, data=data)
        datapath.send_msg(out)
        return

    if (pkt_tcp is not None) and (pkt_ipv4.dst == IP2):

        match = parser.OFPMatch(
            eth_type=0x0800,
            ipv4_src=pkt_ip.src,ipv4_dst=pkt_ip.dst,ip_proto=6,
            tcp_src=pkt_tcp.src_port,tcp_dst=pkt_tcp.dst_port
        )
        actions = [ ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
        return

    data = msg.data
    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

Esercizio 2

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
Net1	*	80	*	TCP
B	D	23	53	TCP
*	E	*	53	UDP
B	*	*	123	*
C	F	23	*	UDP
A	D	25	80	*

La rete Net1 comprende (tra gli altri) gli host A e B.

(a) Definire il bitvector set per ciascun campo.

(b) Assumendo indirizzi IPv4 calcolare la dimensione totale (in bit) del bitvector set. Ripetere il calcolo per IPv6.

(c) Si considerino i pacchetti (H, E, 32, 80, ICMP) e (B, G, 21, 123, TCP). Applicare ad entrambi la procedura di BVLS per identificare gli eventuali matching.

Esercizio 3

Una matrice di commutazione crossbar di uno switch 3×3 usa l'algoritmo Take-a-Ticket (TaT) per lo scheduling delle trasmissioni.

Al tempo $t = 0$ sono presenti i seguenti pacchetti, tutti di lunghezza L, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

- Ingresso 1: 1, 2, 1
- Ingresso 2: 3, 2, 1
- Ingresso 3: 2, 3, 3

(a) Svolgere l'algoritmo TaT fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali ticket sono assegnati. L'algoritmo di assegnazione dei ticket privilegia sempre l'interfaccia di ingresso con **l'identificativo più grande**.

(b) Uno switch 3×3 implementa la seguente variante di TaT. Si i il numero del ticket del pacchetto che inizia la trasmissione su un'interfaccia di uscita. Se ci sono code in ingresso con numero di ticket $\geq i + 2$ per quell'interfaccia, scartano il loro pacchetto e chiedono un ticket per il pacchetto successivo.

Considerando traffico in ingresso distribuito uniformemente e code sempre piene, calcolare il numero di pacchetti scartati per ogni intervallo temporale.

(c) Spiegare cosa si intende per *virtual output queues* e perché sono utili.